

Highlights

Efficient Verification Framework of LTL via Transition Certificates

Yuheng Fan, Zenan Qian, Wang Lin

- We propose transition safety and transition persistence certificates for LTL verification.
- We design an automated counterexample-guided inductive synthesis procedure for transition certificates using both polynomial and neural templates, with solver-guided counterexample refinement.
- On 1D/2D Kuramoto and a two-room temperature benchmark, the method verifies the target LTL properties and consistently reduces synthesis time compared with closure certificate baselines.

Efficient Verification Framework of LTL via Transition Certificates

Yuheng Fan, Zenan Qian and Wang Lin*

Zhejiang Sci-Tech University, Hangzhou, Zhejiang, China

ARTICLE INFO

Keywords:

linear temporal logic
certificate-based verification
cyber-physical systems
CEGIS

ABSTRACT

Context: Verifying Linear Temporal Logic (LTL) properties for discrete-time dynamical systems with continuous state spaces is important in safety-critical software and cyber-physical applications. Existing approaches, such as state-triplet methods and closure certificates, typically suffer from either restrictiveness or high computational cost.

Objectives: This work aims to develop a verification framework that provides greater flexibility and improves efficiency while preserving formal soundness for LTL verification.

Methods: The key insight of our approach is to decompose LTL verification into two types of certificates: transition persistence certificates, which guarantee accepting transitions occur only finitely often, and transition safety certificates, which render accepting transitions unreachable. This design not only reduces the search space significantly compared to closure certificates but also provides greater flexibility than state-triplet methods. We introduce automated synthesis techniques based on polynomial and neural network templates, balancing expressiveness and computational efficiency.

Results: On benchmark case studies, including one-dimensional and two-dimensional Kuramoto oscillator models and a two-room temperature model, the proposed methods successfully synthesize valid certificates and provide formal guarantees. Compared with closure certificate baselines, transition certificates reduce synthesis time.

Conclusion: Transition certificates offer an effective and practical framework for LTL verification of continuous-state dynamical systems. By combining safety and persistence reasoning, the approach improves efficiency and provides greater flexibility relative to prior methods.

1. Introduction

Cyber-Physical Systems (CPS) [3] integrate computational and physical capabilities and are now common in safety-critical applications like autonomous driving and medical devices [30]. For such systems, any failure may result in catastrophic consequences. Linear Temporal Logic (LTL) is a formal language used to define the complex rules for safe operation, such as requiring a timely response or forbidding unsafe conditions [27]. Therefore, the ability to verify that a CPS follows its LTL specification is essential for building trust and ensuring safety in these systems.

Although important, formally verifying LTL specifications for CPS is a significant challenge. The main difficulty arises from the need to handle the infinite-horizon nature of LTL properties [4] together with the infinite state spaces and complex dynamics of physical systems. Traditional model checking techniques based on exhaustive state exploration are not directly applicable to such systems due to their continuous state spaces. Certificate-based verification offers a promising alternative. Instead of enumerating states, one searches for a mathematical function, called a *certificate*, that satisfies certain inequality conditions and thereby serves as a proof that the system meets its specification [29]. For instance, a barrier certificate is a function whose level set separates reachable states from unsafe states, proving safety without computing reachable sets explicitly. A central concern in certificate design is *restrictiveness*: the sufficient conditions imposed by a particular certificate type may be too strict to verify systems that do satisfy the specification. Another concern is *synthesis efficiency*, since certificate parameters must be found automatically and the computational cost grows with the dimensionality of the search space. To address LTL verification specifically, researchers have developed certificate-based methods, including the state-triplet method [36] and closure certificates [25]. The state-triplet method provides an automata-theoretic approach for verifying LTL specifications of dynamical systems. This technique constructs barrier certificates that prevent transitions to accepting states in the automaton representing the negated specification. This approach can be restrictive because it relies on a single proof strategy, namely rendering all accepting states unreachable in the product system via barrier certificates. Consequently, the method structurally fails

*Corresponding author: Wang Lin (linwang@zstu.edu.cn).

✉ linwang@zstu.edu.cn (W. Lin)

ORCID(s):

whenever any accepting state is reachable from the initial set, even if the system satisfies the LTL specification and the accepting transitions would only fire finitely often in practice. The method's strength lies in its direct connection to automata structure, but it may need numerous certificates for complex specifications. Closure certificates offer a generalized framework by operating on pairs of system states through transition invariants. This approach establishes certificates that enforce decrease conditions along trajectories, ensuring accepting states are visited only finitely often. The method provides greater flexibility by allowing more general certificate structures and naturally handles liveness specifications. While these methods are significant, they each have trade-offs. The state-triplet method can be restrictive and closure certificates have low synthesis efficiency because of the large search space they must explore. These limitations show a clear need for a new framework that provides greater flexibility without a high computational cost.

To address this gap, we propose a new framework to verify LTL specifications via proof certificates. Our approach shifts the focus of the LTL verification problem to the transitions of the Nondeterministic Büchi Automaton (NBA) [16]. In our framework, verification is achieved by finding two types of complementary certificates. Transition safety certificates prove that certain accepting transitions never occur, while transition persistence certificates ensure that other accepting transitions happen only a finite number of times. By combining them, we can formally guarantee that no accepting run exists. To find these certificates, we developed an automated synthesis method based on the Counterexample-Guided Inductive Synthesis (CEGIS) framework. This method supports both polynomial templates for efficiency and neural network templates for expressiveness. The main contributions of this paper are summarized as follows:

- We propose a new LTL verification framework for discrete-time dynamical systems with continuous state spaces, which employs transition certificates to reduce the search space, thereby achieving higher computational efficiency compared with closure certificates.
- We introduce automated synthesis methods for transition certificates based on polynomial and neural network templates. These templates provide a balance between expressiveness and computational efficiency, enabling efficient certificate generation for diverse system classes, from linear to nonlinear dynamics.
- We demonstrate the effectiveness of our certificates through case studies, including the Kuramoto oscillator and a two-room temperature model. Experimental results show that transition certificates achieve significant speedups, highlighting their practicality for LTL specifications.

The organization of this paper is as follows: Section 2 reviews related work. Section 3 covers preliminary knowledge. Section 4 introduces our proof certificates for LTL verification and compares with other methods. Section 5 presents synthesis methods. Section 6 demonstrates the effectiveness through case studies. Section 7 concludes the paper.

2. Related Work

Verification of temporal logic properties via proof certificates has seen significant advancements. To validate discrete-time systems against LTL specifications, Wongpiromsarn et al. [36] developed a state-triplet method that uses barrier certificates to block all simple paths from initial states to accepting states in the automaton, requiring enumeration of state triplets (q_m, q'_m, q''_m) along these paths. Murali et al. [25] introduced closure certificates for persistence verification, which establish disjunctively well-founded transition invariants [28] for dynamical systems, and further extended this notion to control closure certificates for ω -regular specifications [24]. Additionally, substantial research has addressed the verification of specific temporal behaviors such as reachability, safety, and avoidance [5–8]. Some adopt neural networks as templates [26, 35, 38] to synthesize certificates for verifying temporal properties, including the simultaneous representation of ranking functions and inductive invariants [15] and the verification of memoryful neural agents against LTL specifications [18].

The abstraction-based approaches [9, 11, 13, 20, 23, 34, 37] lift the CPS model from continuous domain to discrete domain by computing a finite abstraction, e.g., finite transition systems and Markov decision processes (MDPs). We note that abstraction-based techniques have been used in the verification and synthesis of continuous-space dynamical systems against LTL properties such as the results in [17, 19, 22, 31]. These results rely on building a finite state abstraction and then making use of model checking and synthesis techniques [4, 33]. In general, the abstraction-based approaches are computationally demanding and suffer from the curse of dimensionality. Abstraction-free methods, such as proof certificates, can alleviate the computational burden to some extent.

3. Preliminaries

3.1. Notation

We use $\mathbb{N}$ and $\mathbb{R}$ to represent the set of natural numbers and the set of real numbers, respectively. We define:

$$\mathbb{N}_{\sim i} = \{n \in \mathbb{N} \mid n \sim i\}, \quad \text{where } i \in \mathbb{N}, \sim \in \{\geq, \leq, >, <\},$$

$$\mathbb{R}_{\sim i} = \{r \in \mathbb{R} \mid r \sim i\}, \quad \text{where } i \in \mathbb{R}, \sim \in \{\geq, \leq, >, <\}.$$

Given a set S and a set S_1 , we define an indicator function:

$$I_S(x) = \begin{cases} 1, & \text{if } x \in S \\ 0, & \text{if } x \notin S. \end{cases}$$

$|S|$ denotes the cardinality of S and $S \setminus S_1$ denotes $\{x \in S \mid x \notin S_1\}$.

Given a sequence $s = \langle s_0, s_1, \dots \rangle$, where $s_i \in S$ for $i \in \mathbb{N}$:

- $s[i]$ denotes s_i ;
- $s[i : j]$ denotes $\langle s_i, \dots, s_j \rangle$ for $j \geq i$;
- $s[i : \infty]$ denotes $\langle s_i, s_{i+1}, \dots \rangle$.

We say s visits S if there exists $i \in \mathbb{N}$ such that $s_i \in S$. $\text{Inf}(s)$ denotes the set of elements that appear in s infinitely often. S^ω denotes the set of infinite sequences over S , i.e., $S^\omega = \{s \mid s = \langle s_0, s_1, \dots \rangle \text{ and } s_i \in S \text{ for } i \in \mathbb{N}\}$. A function $f : S \rightarrow \mathbb{R}$ is bounded if and only if there exist $a, b \in \mathbb{R}$ such that $a \leq f(x) \leq b$ for all $x \in S$.

3.2. Discrete-time Dynamical System

A discrete-time dynamical system is defined as $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, where $\mathcal{X} \subseteq \mathbb{R}^n$ represents the state set, $\mathcal{X}_0 \subseteq \mathcal{X}$ represents a set of initial states, and $f : \mathcal{X} \rightarrow \mathcal{X}$ represents the transition function. The evolution of the system state is described as follows:

$$x_{i+1} = f(x_i).$$

An infinite sequence of states $\langle x_0, x_1, x_2, \dots \rangle$ is called a *trajectory* of the system if $x_0 \in \mathcal{X}_0$ and $x_{i+1} = f(x_i)$ for $i \in \mathbb{N}$. We use $\mathfrak{T}(\mathfrak{S})$ to represent all *trajectories* on $\mathfrak{S}$. The *predecessor* of x_{i+1} is x_i for $i \in \mathbb{N}$. In a given *trajectory*, x_{i+1} has only one *predecessor*. We use the notation x_i^{pre} to denote the *predecessor* of x_i in a *trajectory*.

3.3. Specification

This paper focuses on the verification of LTL properties for discrete-time dynamical systems. Below we introduce the foundational concepts of safety, persistence, and Nondeterministic Büchi Automata (NBA), which are essential for establishing our framework.

Safety: A system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ is *safe* with respect to unsafe states $\mathcal{X}_u \subseteq \mathcal{X}$ if for any *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, we have $x_i \notin \mathcal{X}_u$ for all $i \in \mathbb{N}$.

Persistence: A system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ is *persistent* with respect to states $\mathcal{X}_{VF} \subseteq \mathcal{X}$ if for any *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, elements of $\mathcal{X}_{VF}$ appear finitely often in τ .

Linear Temporal Logic (LTL) [27]: LTL formulae are defined over a set of atomic propositions AP . A labelling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{AP}$ maps system states to subsets of AP . Given a *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, we denote $\mathfrak{L}(\tau) = \langle \mathfrak{L}(x_0), \mathfrak{L}(x_1), \dots \rangle$ as the corresponding *word*.

The syntax of LTL is:

$$\phi := \top \mid a \mid \neg\phi \mid \phi \wedge \phi \mid X\phi \mid \phi U \phi,$$

where $a \in AP$ is an atomic proposition. Temporal operators finally (F) and always (G) are derived from next (X) and until (U).

Given a *word* $w = \mathfrak{L}(\tau)$ and an LTL formula ϕ , the semantics is defined inductively:

- $w \models \top$ always holds,
- $w \models a$ iff $a \in w[0]$,
- $w \models \phi_1 \wedge \phi_2$ iff $w \models \phi_1$ and $w \models \phi_2$,
- $w \models \neg\phi_1$ iff $w \not\models \phi_1$,
- $w \models X\phi_1$ iff $w[1 : \infty] \models \phi_1$,
- $w \models \phi_1 U \phi_2$ iff there exists $j \in \mathbb{N}$ such that $w[j : \infty] \models \phi_2$ and for all $i \in \mathbb{N}_{<j}$, $w[i : \infty] \models \phi_1$.

We say $\mathfrak{S} \models_{\mathfrak{g}} \phi$ if for any $\tau \in \mathfrak{T}(\mathfrak{S})$, we have $\mathfrak{L}(\tau) \models \phi$.

Nondeterministic Büchi Automaton (NBA) [16]: An NBA is a tuple $(\Sigma, Q, q_0, \delta, Acc)$, where $\Sigma = 2^{AP}$ is the alphabet, Q is a finite set of states, $q_0 \in Q$ is the initial state, $\delta \subseteq Q \times \Sigma \times Q$ is the transition relation, and $Acc \subseteq \delta$ is the set of *accepting transitions*. Given a word $w = \langle w_0, w_1, \dots \rangle \in \Sigma^\omega$, a **run** on w is a sequence $r = \langle (r_0, w_0, r_1), (r_1, w_1, r_2), \dots \rangle$ where $r_0 = q_0$ and $(r_i, w_i, r_{i+1}) \in \delta$ for all $i \in \mathbb{N}$. The *transition-based accepting condition* is that an NBA accepts w if there exists a run where at least one accepting transition occurs infinitely often, i.e., $Inf(r) \cap Acc \neq \emptyset$.

For an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc)$, we define:

$$Acc^{pair} = \{(p, q) \mid \exists \sigma \in \Sigma. (p, \sigma, q) \in Acc\}, \quad (1)$$

$$Acc^{start} = \{p \mid \exists \sigma \in \Sigma, q \in Q. (p, \sigma, q) \in Acc\}, \quad (2)$$

$$Acc^{k,l} = \{(q_k, \sigma, q_l) \mid (q_k, \sigma, q_l) \in Acc\}. \quad (3)$$

We denote $\mathfrak{L}(\mathfrak{S}) = \{\mathfrak{L}(\tau) \mid \tau \in \mathfrak{T}(\mathfrak{S})\}$ as the *language* of $\mathfrak{S}$, and $\mathfrak{L}(\mathcal{A})$ as the language accepted by $\mathcal{A}$. To verify $\mathfrak{S} \models_{\mathfrak{g}} \phi$, we construct an NBA $\mathcal{A}$ for $\neg\phi$ and show $\mathfrak{L}(\mathfrak{S}) \cap \mathfrak{L}(\mathcal{A}) = \emptyset$.

3.4. Proof Certificates

Barrier certificates [29] and closure certificates [25] are used for verifying LTL properties [25, 36], respectively. We provide their definitions and related conclusions from prior work.

Definition 3.1 (barrier certificate [29]). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ and $\mathcal{X}_u \subseteq \mathcal{X}$, a bounded function $B(x) : \mathcal{X} \rightarrow \mathbb{R}$ is a *barrier certificate* with respect to $\mathcal{X}_u$ if for all $x_0 \in \mathcal{X}_0$, $x_u \in \mathcal{X}_u$, $x \in \mathcal{X}$, and $x' = f(x)$, the following conditions hold:

$$B(x_0) \geq 0, \quad (4)$$

$$B(x_u) < 0, \quad (5)$$

$$B(x) \geq 0 \Rightarrow B(x') \geq 0. \quad (6)$$

Theorem 3.2 (Safety [29]). *The existence of a barrier certificate $B(x)$ implies that the system $\mathfrak{S}$ is safe with respect to $\mathcal{X}_u$.*

The state-triplet approach[36] uses barrier certificates to verify LTL properties. Given an LTL specification ϕ , it proves that accepting states are unreachable in the NBA for $\neg\phi$ by finding barrier certificates to ensure that runs on $\mathfrak{L}(\mathfrak{S})$ will never visit accepting states, verifying $\mathfrak{S} \models_{\mathfrak{g}} \phi$. The state-triplet method needs to find barrier certificates for each simple path, with a certificate corresponding to a specific state triplet to individually block a simple path. The state-triplet method introduces restrictiveness since it cannot be applied to systems that satisfy the LTL specification but cannot be proven safe under these restrictive constraints.

Definition 3.3 (closure certificate [25]). Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a set of atomic propositions AP , a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{AP}$ and an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, Acc)$ represent $\neg\phi$. A bounded function $T : \mathcal{X} \times Q \times \mathcal{X} \times Q \rightarrow \mathbb{R}$ is a closure certificate for ϕ if there exists a value $\epsilon \in \mathbb{R}_{>0}$ such that for all states $x, y, y' \in \mathcal{X}$, $x' = f(x)$, states $q_i, q_j \in Q$, and $q'_i \in \delta(q_i, \mathfrak{L}(x))$, we have:

$$T((x, q_i), (x', q'_i)) \geq 0, \quad (7)$$

$$T((x', q'_i), (y, q_j)) \geq 0 \Rightarrow T((x, q_i), (y, q_j)) \geq 0. \quad (8)$$

and for all states $x_0 \in \mathcal{X}_0$, and $\ell, \ell' \in \text{Acc}$, we have:

$$\begin{aligned} T((x_0, q_0), (y, \ell)) &\geq 0 \wedge T((y, \ell), (y', \ell')) \geq 0 \\ \Rightarrow T((x_0, q_0), (y', \ell')) &\leq T((x_0, q_0), (y, \ell)) - \epsilon. \end{aligned} \quad (9)$$

Theorem 3.4 (LTL Verification [25]). *Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a set of atomic propositions AP , a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{AP}$ and an LTL formula ϕ . Let an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, \text{Acc})$ represent $\neg\phi$. The existence of a closure certificate satisfying conditions (7)-(9) implies that $\mathfrak{S} \models_{\mathfrak{L}} \phi$.*

Closure certificates are defined by characterizing an over-approximation of the transition closure along with a decrease condition, ensuring that transitions in certain regions are visited only finitely often, and are used for LTL verification. Closure certificates (3.3) are defined on $\mathcal{X} \times Q \times \mathcal{X} \times Q$, creating high-dimensional search spaces with heavy computational burdens.

3.5. Problem Statement

Problem. Given a discrete-time dynamical system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a set of atomic propositions AP , a labeling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{AP}$, and an LTL formula ϕ , verify that $\mathfrak{S} \models_{\mathfrak{L}} \phi$ holds.

In this paper, we propose a new approach to address the LTL verification problem.

4. Transition Certificates

In this section, we propose a new type of proof certificates, called transition certificates, to verify that a system $\mathfrak{S}$ satisfies an LTL formula ϕ . The core idea of transition certificates is to reduce the verification problem to analyzing the behavior of accepting transitions in the NBA $\mathcal{A}$ for $\neg\phi$. By the definition of the transition-based acceptance condition, a word is accepted by $\mathcal{A}$ if and only if accepting transitions occur infinitely often along one of its runs. Therefore, to prove $\mathfrak{S} \models_{\mathfrak{L}} \phi$, it suffices to show that for all trajectories of $\mathfrak{S}$, every accepting transition in $\mathcal{A}$ occurs at most finitely many times. We introduce two types of transition certificates: *transition safety certificates* and *transition persistence certificates*. The two certificates address the problem from fundamentally different angles and concern the set $\text{Acc}^{\text{start}}$, which comprises the starting states of all accepting transitions in $\mathcal{A}$ (cf. equation 2).

- A *transition safety certificate* proves that the system's trajectories can never reach $\text{Acc}^{\text{start}}$, thereby guaranteeing that related accepting transitions **never occur**.
- A *transition persistence certificate* allows the system to reach $\text{Acc}^{\text{start}}$ but proves that related accepting transitions can only be taken **finitely often**.

The high-level intuition is as follows. Given an NBA for $\neg\phi$, we examine each accepting transition individually. If the source state of an accepting transition is unreachable from the initial product state, we certify this with a safety argument, using a function that is non-negative initially and negative at the source state, while preserving non-negativity along trajectories. If the source state is reachable, we take a different approach, showing that the system's dynamics force a strictly positive decrease in a bounded, non-negative function each time the accepting transition could fire, so it can only fire finitely many times. By covering all accepting transitions through one mechanism or the other, we guarantee that no accepting run exists.

4.1. Transition Safety Certificate

The objective of a transition safety certificate is to prevent any accepting transition from occurring. To achieve this, we verify that accepting transitions never occur by showing that $\text{Acc}^{\text{start}}$ is unreachable in the product of the system and the automaton.

Definition 4.1 (transition safety certificate). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{AP}$ and an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, \text{Acc})$. Construct the product system $\mathfrak{S} \times \mathcal{A} = (\mathcal{Y}, \mathcal{Y}_0, F)$, where $\mathcal{Y} = \{(x, q) \mid x \in \mathcal{X}, q \in Q\}$, $\mathcal{Y}_0 = \{(x_0, q_0) \mid x_0 \in \mathcal{X}_0\}$, and $F(x, q) = \{(x', q') \mid x' = f(x) \wedge (q, \mathfrak{L}(x), q') \in \delta\}$. For $q_k \in \text{Acc}^{\text{start}}$, define the unsafe

set $\mathcal{Y}_u = \{(x, q_k) \mid x \in \mathcal{X}\}$. A bounded function $B_k(x, q) : \mathcal{X} \times \mathcal{Q} \rightarrow \mathbb{R}$ is a *transition safety certificate* with respect to $q_k \in \text{Acc}^{\text{start}}$ if for all $(x, q) \in \mathcal{Y}$, $(x_0, q_0) \in \mathcal{Y}_0$, $(x', q') \in F(x, q)$, we have:

$$B_k(x_0, q_0) \geq 0, \quad (10)$$

$$B_k(x, q_k) < 0, \quad (11)$$

$$B_k(x, q) \geq 0 \Rightarrow B_k(x', q') \geq 0. \quad (12)$$

Intuitively, condition (10) guarantees that the certificate value is non-negative at the initial state of the product system; condition (11) ensures that the certificate becomes negative precisely on the unsafe set $\mathcal{Y}_u^k$ associated with q_k , thereby marking states where accepting transitions could originate; condition (12) implies that once the certificate value is non-negative at a state, it remains non-negative along any trajectory of the product system, thus preserving the safety invariant.

Collectively, these conditions ensure that the certificate B_k guarantees q_k is unreachable, preventing all accepting transitions from q_k . The following lemma formalizes this guarantee. Its proof is provided in Appendix A.

Lemma 4.2. *If a transition safety certificate B_k with respect to $q_k \in \text{Acc}^{\text{start}}$ can be found, then all runs of words in $\mathfrak{L}(\mathfrak{S})$ never visit state q_k , thereby preventing all accepting transitions starting from q_k .*

The next theorem follows directly from applying the above lemma to every state in $\text{Acc}^{\text{start}}$. If a transition safety certificate can be found for each state in $\text{Acc}^{\text{start}}$, then no accepting transition can be taken, ensuring the LTL specification holds.

Theorem 4.3. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{AP}$ and an NBA $\mathcal{A} = (2^{AP}, \mathcal{Q}, q_0, \delta, \text{Acc})$ for $\neg\phi$. If for any $q_k \in \text{Acc}^{\text{start}}$, a transition safety certificate B_k can be found, then it can be guaranteed that $\mathfrak{S} \models_{\mathfrak{L}} \phi$.*

Proof. By leveraging Lemma 4.2, all start states of accepting transitions are unreachable. Consequently, all accepting transitions never occur, ensuring that the system's language is not accepted and thereby guaranteeing $\mathfrak{S} \models_{\mathfrak{L}} \phi$. $\square$

4.2. Transition Persistence Certificate

In contrast, the *transition persistence certificate* offers an alternative approach. Instead of completely preventing the system from reaching states where accepting transitions may happen, this certificate proves that even if such states are reached, the corresponding accepting transitions can occur only a finite number of times. This is achieved by requiring a decrease in a certificate value each time an accepting transition is taken.

Definition 4.4 (transition persistence certificate). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{AP}$ and an NBA $\mathcal{A} = (2^{AP}, \mathcal{Q}, q_0, \delta, \text{Acc})$. For $(k, l) \in \text{Acc}^{\text{pair}}$, define the indicator function $I_{k,l}(x) = 1$ if $(q_k, \mathfrak{L}(x), q_l) \in \text{Acc}^{k,l}$, and 0 otherwise. A bounded function $B_{k,l}(x) : \mathcal{X} \rightarrow \mathbb{R}$ is a *transition persistence certificate* with respect to $\text{Acc}^{k,l}$ if there exists $\epsilon > 0$ such that for all $x_0 \in \mathcal{X}_0$, $x \in \mathcal{X}$, $x' = f(x)$, we have:

$$B_{k,l}(x_0) \geq 0, \quad (13)$$

$$B_{k,l}(x) \geq 0 \Rightarrow B_{k,l}(x') \geq 0, \quad (14)$$

$$B_{k,l}(x) \geq B_{k,l}(x') + I_{k,l}(x)\epsilon. \quad (15)$$

We emphasize two important aspects of Definition 4.4. First, each certificate $B_{k,l}$ is constructed for a *specific* accepting transition pair $(k, l) \in \text{Acc}^{\text{pair}}$; different pairs may require different certificates. Second, $B_{k,l}(x) : \mathcal{X} \rightarrow \mathbb{R}$ is a function of the system state x alone and does not depend on the current automaton state. This design is sound because the indicator $I_{k,l}(x)$ captures a *necessary condition* for the accepting transition $(q_k, \mathfrak{L}(x), q_l)$ to occur. Regardless of which automaton state a particular run occupies at time step i , whenever $I_{k,l}(x_i) = 0$ the transition from q_k to q_l cannot be taken. By bounding the number of visits to the enabling region $\{x \mid I_{k,l}(x) = 1\}$ along system trajectories, the certificate provides an upper bound on the number of times the accepting transition can actually occur in *any* run.

This design introduces a degree of conservatism. Because $B_{k,l}$ constrains the total number of time steps during which $I_{k,l}(x) = 1$ without regard to the current automaton state, it limits visits to the enabling region even when the system occupies a state other than q_k . As a consequence, a valid persistence certificate may fail to exist even when the LTL specification is satisfied, if the trajectory frequently enters the enabling region while the automaton is in a

state other than q_k . This conservatism is the price of working with a function over $\mathcal{X}$ rather than the full product space $\mathcal{X} \times \mathcal{Q}$, and it is analogous to the over-approximation inherent in trajectory-based Lyapunov arguments.

Intuitively, condition (13) guarantees that the certificate $B_{k,l}$ is non-negative at the initial state of the system; condition (14) ensures that $B_{k,l}$ remains non-negative along any trajectory of the system; condition (15) implies that $B_{k,l}$ decreases by at least ϵ whenever the state's label matches an accepting transition from q_k to q_l , and remains non-increasing otherwise.

Since accepting transitions from q_k to q_l can only occur when $(q_k, \mathfrak{L}(x), q_l) \in \text{Acc}^{k,l}$, the number of such transitions is bounded by $B_{k,l}(x_0)/\epsilon$, ensuring finite occurrence. This yields the following lemma. Its proof is provided in the Appendix A.

Lemma 4.5. *If a transition persistence certificate $B_{k,l}$ can be found, then all runs of words in $\mathfrak{L}(\mathfrak{S})$ make accepting transitions from q_k to q_l happen finitely often.*

The next theorem is obtained by applying the lemma to every accepting transition pair.

Theorem 4.6. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{AP}$ and an NBA $\mathcal{A} = (2^{AP}, \mathcal{Q}, q_0, \delta, \text{Acc})$ for $\neg\phi$. if for any $(k, l) \in \text{Acc}^{\text{pair}}$, a corresponding transition persistence certificate $B_{k,l}$ can be found, then it can be guaranteed that $\mathfrak{S} \models_{\mathfrak{L}} \phi$.*

Proof. By leveraging Lemma 4.5, all accepting transitions occur only finitely often, which ensures that the system's language is not accepted, thereby guaranteeing $\mathfrak{S} \models_{\mathfrak{L}} \phi$. $\square$

While both the methods established by Theorem 4.3 and Theorem 4.6 can be used to verify LTL properties, the two types of certificates are complementary and, when combined, significantly expand the range of systems that can be verified. In some scenarios, requiring the system to completely avoid certain states might be too strict, making transition safety certificates impossible to find. However, the system might still satisfy the property if the accepting transitions occur only finitely often, a condition that can be certified by a transition persistence certificate. Conversely, consider verifying $\mathbf{G}\neg a$ using the NBA for $\mathbf{F}a$, where $\text{Acc}^{0,0} = \{(q_0, \emptyset, q_0), (q_0, \{a\}, q_0)\}$. For the persistence certificate $B_{0,0}$, condition (15) requires $B_{0,0}(x) \geq B_{0,0}(f(x)) + \epsilon$ for any x where an accepting transition is enabled. This would demand an infinite descent in the bounded certificate value, which is impossible, making it impossible to satisfy the conditions for a transition persistence certificate, and thus such a certificate cannot exist. Yet, a transition safety certificate can successfully verify the property by proving that state q_0 (the start of the accepting transitions) is unreachable in the product system. Thus, providing both certificates covers a wider range of system behaviors, enhancing the method's applicability. We now present the main theorem for LTL verification. For each accepting transition start state in the NBA, we use either a transition safety certificate or transition persistence certificates to prove accepting transitions from that state occur finitely often.

Theorem 4.7 (LTL verification). *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow \Sigma$ and an NBA $\mathcal{A} = (\Sigma, \mathcal{Q}, q_0, \delta, \text{Acc})$ representing the negation of an LTL formula ϕ . For each $q_k \in \text{Acc}^{\text{start}}$, if we can find either:*

1. *A transition safety certificate B_k with respect to q_k , or*
2. *Transition persistence certificates $B_{k,l}$ for all $(k, l) \in \text{Acc}^{\text{pair}}$ where q_k is the start state,*

then it can be guaranteed that $\mathfrak{S} \models_{\mathfrak{L}} \phi$.

Proof. Given an accepting transition start state $q_k \in \text{Acc}^{\text{start}}$:

1. If a transition safety certificate B_k is found, by Lemma (4.2), all runs never visit state q_k , thereby preventing all accepting transitions starting from q_k .
2. For each $(k, l) \in \text{Acc}^{\text{pair}}$ where the start state is q_k , if a transition persistence certificate $B_{k,l}$ is found, by Lemma (4.5), all runs make accepting transitions from q_k to q_l happen finitely often.

This ensures that all accepting transitions in Acc occur only finitely often. By the definition of NBA acceptance, a word w is accepted if and only if there exists a run where accepting transitions occur infinitely often, i.e., $\text{Inf}(r) \cap \text{Acc} \neq \emptyset$. Since all accepting transitions occur only finitely often, for any word $w \in \mathfrak{L}(\mathfrak{S})$ and any run r on w , we have $\text{Inf}(r) \cap \text{Acc} = \emptyset$. Therefore, no word in $\mathfrak{L}(\mathfrak{S})$ is accepted by automaton $\mathcal{A}$, implying $\mathfrak{L}(\mathfrak{S}) \cap \mathfrak{L}(\mathcal{A}) = \emptyset$. Since $\mathfrak{L}(\mathcal{A}) = \mathfrak{L}(\neg\phi)$, we have $\mathfrak{L}(\mathfrak{S}) \subseteq \mathfrak{L}(\phi)$, and consequently $\mathfrak{S} \models_{\mathfrak{L}} \phi$. $\square$

One structural aspect of Theorem 4.7 warrants clarification. For each $q_k \in Acc^{start}$, the verification must commit to a single strategy: either supply a transition safety certificate B_k that simultaneously prevents all accepting transitions originating from q_k , or supply individual persistence certificates $B_{k,l}$ for every pair $(k, l) \in Acc^{pair}$ with start state q_k . The two strategies cannot be mixed for the same q_k , since a safety certificate B_k makes a global argument that q_k is entirely unreachable, which already prevents all accepting transitions from q_k and leaves no residual role for a persistence certificate at that state.

In summary, the combination of transition safety certificates and transition persistence certificates offers a more flexible verification framework. We emphasize that the novelty of our approach does not lie in merely combining barrier-like and ranking-like arguments, which are well-established proof techniques. Rather, the key contribution is the *transition-level decomposition* of the verification problem. Instead of constructing a single monolithic certificate over the entire product state space, we decompose the task at the granularity of individual accepting transitions. For each accepting transition start state q_k , we independently select the most suitable proof strategy (safety or persistence) based on the local structure of the automaton and the system dynamics. This per-transition decomposition yields certificates that each operate over a smaller domain with simpler conditions, reducing both the dimensionality of the synthesis problem and the restrictiveness of the sufficient conditions. The resulting framework maintains a smaller search space compared to closure certificates while providing two distinct mechanisms to ensure that accepting transitions occur only finitely often, ultimately proving that the system satisfies its LTL specification.

5. Certificate Synthesis

This section develops automated synthesis techniques for transition certificates. We leverage the Counterexample-Guided Inductive Synthesis (CEGIS) framework [32], a well-established paradigm in barrier certificate synthesis, to iteratively refine candidate solutions. By alternating between candidate generation and formal verification, this approach ensures that the discovered certificates are valid and proves that the system meets the specified LTL properties. Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ and an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, Acc)$ representing $\neg\phi$, we adopt a uniform notation where B_k represents the transition safety certificate template for states $q_k \in Acc^{start}$, and $B_{k,l}$ represents the transition persistence certificate template for $Acc^{k,l}$. To find transition certificates, we present CEGIS frameworks for polynomial and neural network templates, respectively. The synthesis proceeds in two alternating phases: *candidate generation* and *verification*.

5.1. Polynomial Template CEGIS

5.1.1. Transition Safety Certificates

We define a parametric polynomial template:

$$B_k(x, q; \mathbf{c}) = \sum_{i=1}^m c_i p_i(x, q), \quad (16)$$

where $x \in \mathcal{X}$, $q \in Q$, $\{p_1, \dots, p_m\}$ is a user-defined basis of monomials over $\mathcal{X} \times Q$ with degree at most d , and $\mathbf{c} = (c_1, \dots, c_m) \in \mathbb{R}^m$ are unknown coefficients. Our goal is to find $\mathbf{c}$ such that $B_k(\cdot, \cdot; \mathbf{c})$ satisfies conditions (10), (11), and (12).

Candidate Generation. Based on certificate conditions, we encode the constraint for all points of the sampled dataset. For transition safety certificates, the constraint is:

$$\bigwedge_{(x,q) \in D_0} B_k(x, q; \mathbf{c}) \geq 0 \quad (17)$$

$$\wedge \bigwedge_{(x,q) \in D_u} B_k(x, q; \mathbf{c}) < 0 \quad (18)$$

$$\wedge \bigwedge_{\substack{(x,q) \in D_{\mathcal{X}} \\ (q, \mathfrak{A}(x), q') \in \delta}} [B_k(x, q; \mathbf{c}) \geq 0 \Rightarrow B_k(f(x), q'; \mathbf{c}) \geq 0], \quad (19)$$

where D_0 , D_u , $D_{\mathcal{X}}$ are the datasets sampled from $\mathcal{X}_0 \times \{q_0\}$, $\mathcal{X} \times \{q_k\}$, and $\mathcal{X} \times Q$, respectively. The coefficient synthesis involves logical implications (disjunctive constraints) between linear inequalities. We employed Z3 [10] as an SMT

solver to directly handle the logical structure. If the constraint is satisfiable with a solution $\hat{\mathbf{c}}$, define the candidate certificate as $\hat{B}_k = B_k(\cdot, \cdot; \hat{\mathbf{c}})$.

Verification. To verify $\hat{B}_k$ is valid, we formulate the negation of conditions and verify they are unsatisfiable. For transition safety certificates, we check:

$$x \in \mathcal{X}_0 \wedge \hat{B}_k(x, q_0) < 0, \quad (20)$$

$$x \in \mathcal{X} \wedge \hat{B}_k(x, q_k) \geq 0, \quad (21)$$

$$x \in \mathcal{X} \wedge (q, \mathfrak{Q}(x), q') \in \delta \wedge \hat{B}_k(x, q) \geq 0 \wedge \hat{B}_k(f(x), q') < 0. \quad (22)$$

We choose dReal [14] as our SMT verifier to check these constraints. If the constraints are satisfiable (i.e., counterexamples exist), we add them to D_0 , D_u , or $D_{\mathcal{X}}$ accordingly. The synthesis process then iterates until no counterexamples are found, yielding a valid transition safety certificate.

5.1.2. Transition Persistence Certificates

We define a parametric polynomial template:

$$B_{k,l}(x; \mathbf{c}) = \sum_{i=1}^m c_i p_i(x), \quad (23)$$

where $x \in \mathcal{X}$, $\{p_1, \dots, p_m\}$ are monomials over $\mathcal{X}$ and $\mathbf{c} \in \mathbb{R}^m$ are unknown coefficients. We seek $\mathbf{c}$ and $\epsilon > 0$ satisfying conditions (13), (14), and (15).

Candidate Generation. Based on certificate conditions, we encode the constraint for all points of the sampled dataset. For transition persistence certificates, the constraint is:

$$\bigwedge_{x \in D_0} B_{k,l}(x; \mathbf{c}) \geq 0 \quad (24)$$

$$\wedge \bigwedge_{x \in D_{\mathcal{X}}} [B_{k,l}(x; \mathbf{c}) \geq 0 \Rightarrow B_{k,l}(f(x); \mathbf{c}) \geq 0] \quad (25)$$

$$\wedge \bigwedge_{\substack{x \in D_{\mathcal{X}} \\ (q_k, \mathfrak{Q}(x), q_l) \notin Acc^{k,l}}} B_{k,l}(x; \mathbf{c}) \geq B_{k,l}(f(x); \mathbf{c}) \quad (26)$$

$$\wedge \bigwedge_{\substack{x \in D_{\mathcal{X}} \\ (q_k, \mathfrak{Q}(x), q_l) \in Acc^{k,l}}} B_{k,l}(x; \mathbf{c}) \geq B_{k,l}(f(x); \mathbf{c}) + \epsilon, \quad (27)$$

where $D_0, D_{\mathcal{X}}$ are the datasets sampled from $\mathcal{X}_0$ and $\mathcal{X}$, respectively. We solve the coefficient synthesis problem using the Z3 solver [10]. If satisfiable with solution $(\hat{\mathbf{c}}, \hat{\epsilon})$, define the candidate transition persistence certificate as $\hat{B}_{k,l} = B_{k,l}(\cdot; \hat{\mathbf{c}})$.

Verification. To verify $\hat{B}_{k,l}$ is valid, we encode constraints of the negation of conditions and verify they are unsatisfiable. For transition persistence certificates, we check:

$$x \in \mathcal{X}_0 \wedge \hat{B}_{k,l}(x) < 0, \quad (28)$$

$$x \in \mathcal{X} \wedge \hat{B}_{k,l}(x) \geq 0 \wedge \hat{B}_{k,l}(f(x)) < 0, \quad (29)$$

$$x \in \mathcal{X} \wedge (q_k, \mathfrak{Q}(x), q_l) \in Acc^{k,l} \wedge \hat{B}_{k,l}(x) < \hat{B}_{k,l}(f(x)) + \hat{\epsilon}, \quad (30)$$

$$x \in \mathcal{X} \wedge (q_k, \mathfrak{Q}(x), q_l) \notin Acc^{k,l} \wedge \hat{B}_{k,l}(x) < \hat{B}_{k,l}(f(x)). \quad (31)$$

We use dReal [14] to check these constraints. If the constraints are satisfiable, we add them to D_0 or $D_{\mathcal{X}}$ accordingly. The synthesis process then iterates until no counterexamples are found, yielding a valid transition persistence certificate.

5.2. Neural Template CEGIS

Since neural networks have the capability to approximate arbitrary functions, using them as templates for certificates offers advantages over polynomials [38]. Assume that $x \in \mathbb{R}^n$ represents the n -dimensional input vector of

the neural network, $L - 1$ represents the number of hidden layers, and y is the scalar output of the neural network with respect to the input x . Thus, the feedforward neural network can be expressed with the following structure:

$$\begin{cases} z_l = \mathbf{W}_l x_{l-1} + \mathbf{b}_l, & 0 < l \leq L \\ x_l = \sigma(z_l), & 0 < l \leq L \\ y = x_L \end{cases} \quad (5)$$

where,

- z_l denotes the output vector of the linear operation in the l -th layer;
- $\mathbf{W}_l$ and $\mathbf{b}_l$ denote the weight matrix and bias vector for the l -th layer of the neural network, respectively;
- σ denotes the activation function. Common activation functions include the Tanh function, Sigmoid function, ReLU function.

Candidate Generation. Based on certificate conditions, a loss function is designed for the training dataset. For transition safety certificates, the loss function is:

$$\begin{aligned} \mathcal{L}_B = & \sum_{(x,q) \in D_0} \text{ReLU}(-B_k(x, q; \theta)) \\ & + \sum_{(x,q) \in D_u} \text{ReLU}(B_k(x, q; \theta)) \\ & + \sum_{\substack{(x,q) \in D_{\mathcal{X}} \\ (q, \mathfrak{Q}(x), q') \in \delta}} \text{ReLU}(B_k(x, q; \theta)) \cdot \text{ReLU}(-B_k(f(x), q'; \theta)), \end{aligned} \quad (32)$$

where $D_0, D_u, D_{\mathcal{X}}$ are datasets sampled from $\mathcal{X}_0 \times \{q_0\}$, $\mathcal{X} \times \{q_k\}$, and $\mathcal{X} \times Q$, respectively. The terms in the loss function offer intuitive measures of the extent to which the neural network violates the certificate conditions.

For transition persistence certificates, the loss function is:

$$\mathcal{L}_B = \sum_{\substack{x \in D_{\mathcal{X}} \\ (q_k, \mathfrak{Q}(x), q_l) \in \text{Acc}^{k,l}}} \text{ReLU}(B_{k,l}(f(x); \theta) + \epsilon - B_{k,l}(x; \theta)) \quad (33)$$

$$+ \sum_{\substack{x \in D_{\mathcal{X}} \\ (q_k, \mathfrak{Q}(x), q_l) \notin \text{Acc}^{k,l}}} \text{ReLU}(B_{k,l}(f(x); \theta) - B_{k,l}(x; \theta)), \quad (34)$$

where $D_{\mathcal{X}} \subseteq \mathcal{X}$ is the dataset sampled from $\mathcal{X}$.

To generate a neural certificate candidate, we employ gradient descent techniques to minimize the loss function. When the loss decreases to 0, it indicates that the learned neural network can be a candidate certificate. In addition, the activation functions differ. The transition safety certificate employs the ReLU activation function, while the transition persistence certificate uses the squared activation function. This design eliminates the need to verify non-negativity conditions during verification. We define the candidate transition safety certificate as $\hat{B}_k = B_k(\cdot, \cdot; \hat{\theta})$ and the candidate transition persistence certificate to be $\hat{B}_{k,l} = B_{k,l}(\cdot; \hat{\theta})$.

Verification. To verify the candidate neural certificate is valid, we formulate the negation of conditions and use dReal [14] to verify they are unsatisfiable. For transition safety certificates, we check constraints (20), (21), and (22). For transition persistence certificates, we verify constraints (30) and (31). If counterexamples are found, they are added to the training set accordingly, and the synthesis process iterates until no counterexamples remain.

We choose dReal [14] as our SMT verifier, which ensures the correctness of its unsat decisions. Therefore, when dReal returns unsat for the given formula, it confirms that no solution exists within the specified precision, and the candidate certificate is valid. Otherwise, it means that dReal has found a counterexample that violates the certificate conditions. These counterexamples are added to the sample sets and the candidate generation phase is repeated. The process iterates until all verification queries are unsatisfiable (within the δ -completeness threshold of dReal), yielding a valid certificate.

We briefly comment on completeness. For polynomial templates applied to systems with polynomial dynamics and semialgebraic constraints, the certificate conditions can in principle be discharged via Positivstellensatz-type arguments, which guarantee that a sufficient polynomial certificate of high enough degree exists whenever the property holds under mild regularity conditions. In this setting, iterating over templates of increasing degree provides a theoretical path to completeness, though the required degree may be large in the worst case. For neural network templates, the situation differs: failure of the CEGIS loop to converge indicates that the current architecture lacks the expressiveness to represent a valid certificate, not that no certificate exists. In such cases, enlarging the network, changing the activation function, or switching to a polynomial template of higher degree may resolve the issue.

6. Experimental Evaluation

We evaluate the proposed transition certificate synthesis methods on two benchmark systems to demonstrate their effectiveness in formal LTL verification and their computational advantages. We compare against three baselines: the state-triplet method [36], closure certificates [25], and neural closure certificates [26]. For the first benchmark, the Kuramoto Oscillator model (covering both 1D and 2D scenarios), we verify the system against specified LTL properties. For the second benchmark, the two-room temperature model, we demonstrate the versatility of our approach by synthesizing transition certificates to satisfy complex LTL requirements and highlight their complementary advantages in handling diverse temporal specifications. All experiments are conducted on Ubuntu 22.04.3 LTS (Intel i7-9750H, 8 GB RAM). During candidate generation, Z3 addresses the constrained optimization for polynomial templates, whereas neural network templates are trained using the Adam optimizer (learning rate 0.001, 1000 iterations) to minimize the loss function. For formal verification, we employ dReal (precision 0.0001) to handle the transcendental sin functions in the Kuramoto Oscillator model, while Z3 is utilized for the temperature model.

6.1. Kuramoto Oscillator

6.1.1. One-Dimensional System

The first benchmark is a Kuramoto oscillator system, formally defined as $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ with dynamics adapted from [1]. The system operates within the state space $\mathcal{X} = [0, 2\pi]$, starting from an initial region $\mathcal{X}_0 = [\frac{4\pi}{9}, \frac{5\pi}{9}]$. To evaluate LTL verification, we specify an unsafe region $\mathcal{X}_u = [\frac{7\pi}{9}, \frac{8\pi}{9}]$ within the temporal properties. The discrete-time transition function f is given by:

$$f(x) = x + t_s \Omega + t_s K \sin(-x) - 0.532x^2 + 1.69, \quad (35)$$

where x represents the oscillator phase. The system parameters are set as follows: sampling time $t_s = 0.1$, natural frequency $\Omega = 0.01$, and coupling strength $K = 0.0006$.

We consider the LTL specification $\phi = \mathbf{G}\neg a$ over the atomic proposition set $AP = \{a\}$, with the labeling function defined as $\mathfrak{L}(x) = \{a\}$ for $x \in \mathcal{X}_u$ and $\mathfrak{L}(x) = \emptyset$ otherwise. The negated specification, $\neg\phi = \mathbf{F}a$, is characterized by the NBA illustrated in Figure 1. This automaton incorporates accepting transitions (highlighted in red) are depicted as self-loops at state $q_0 \in Acc^{\text{start}}$.

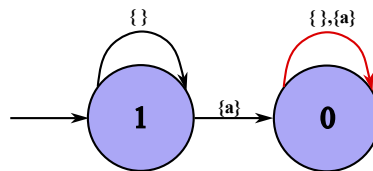


Figure 1: NBA representing the negated specification $\mathbf{F}a$. Accepting transitions (highlighted in red) are depicted as self-loops at state q_0 .

Based on the definition of transition safety certificates, we implement a polynomial-based CEGIS framework to synthesize certificates that guarantee the satisfaction of the specified LTL property. The polynomial template is structured as follows:

$$B_0(x, q; \mathbf{c}) = \sum_{i=0}^4 c_i x^i + \sum_{j=1}^3 c_{4+j} q^j + c_8 x q.$$

The synthesis process begins by collecting 30 samples from region D_0 , 600 from D_u , and 1200 from $D_{\mathcal{X}}$. The candidate generation phase is formulated as a constraint satisfaction problem and solved using the Z3 solver. Given that the system dynamics involve the transcendental sin function, which lies beyond the decision procedures of Z3, we employ the dReal solver during the verification phase to identify counterexamples. The complete CEGIS loop converges in 1.76 seconds, successfully yielding a valid transition certificate:

$$B_0(x, q) = -1 + 2q^2 - \frac{7}{16}xq.$$

In addition to polynomial templates, we synthesize a neural transition certificate $B_0(x, q; \theta)$ using the CEGIS framework. The certificate is parameterized by a feedforward neural network with a 2-10-1 architecture, featuring two input nodes for the pair (x, q) , a hidden layer of 10 neurons with ReLU activation, and a single linear output. The training process is initialized with a dataset of 250 samples (40 from D_0 , 70 from D_u , and 140 from $D_{\mathcal{X}}$). During each iteration, the dReal solver is employed to identify any points violating the certificate conditions. These identified counterexamples are then incorporated into the training set to progressively refine the candidate certificate. The complete CEGIS loop converges in 6.81 seconds. The valid certificates obtained during the process are provided in the appendix.

For comparison, the state-triplet method [36] synthesizes a barrier certificate for the relevant simple path in 0.10 seconds under a degree-4 polynomial template. The certificate $B(x) = -4.5x^2 + 2x^3$ blocks the accepting state via the state-triplet barrier conditions; one accepting edge is discharged trivially because its label region is empty.

6.1.2. Two-Dimensional System

The benchmark is further extended to a two-dimensional scenario with state space $\mathcal{X} = [0, \frac{8\pi}{9}]^2$ and initial region $\mathcal{X}_0 = [0, \frac{\pi}{9}]^2$. In this case, the unsafe region $\mathcal{X}_u$ is defined as the boundary corridors of the state space:

$$\mathcal{X}_u = \left([0, \frac{8\pi}{9}] \times [\frac{5\pi}{6}, \frac{8\pi}{9}]\right) \cup \left([\frac{5\pi}{6}, \frac{8\pi}{9}] \times [0, \frac{8\pi}{9}]\right). \quad (36)$$

The coupled system dynamics f are governed by:

$$f(x_1, x_2) = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + t_s \begin{bmatrix} \Omega + 1.69 \\ \Omega + 1.69 \end{bmatrix} + K t_s \begin{bmatrix} \sin(x_2 - x_1) \\ \sin(x_1 - x_2) \end{bmatrix} - 0.532 t_s \begin{bmatrix} x_1^2 \\ x_2^2 \end{bmatrix}. \quad (37)$$

To maintain consistency with the 1D analysis, we define the labeling function such that $\mathfrak{L}(x_1, x_2) = \{a\}$ if $(x_1, x_2) \in \mathcal{X}_u$ and $\emptyset$ otherwise. The verification is performed against the same LTL specification $\phi = \mathbf{G}\neg a$ using the NBA illustrated in Figure 1.

For the two-dimensional system, we first employ a polynomial template $B_0(x_1, x_2, q; \mathbf{c})$. The synthesis process utilizes a dataset of 30 samples from D_0 , 729 from D_u , and 1,458 from $D_{\mathcal{X}}$. Within 1.72 seconds, the CEGIS loop yields a valid transition certificate:

$$B_0(x_1, x_2, q) = -1 + \frac{1}{8}x_1x_2 + \frac{3}{2}q^2 - \frac{3}{32}qx_1^2 - \frac{153}{512}x_2q. \quad (38)$$

Furthermore, we implement a neural network template with a 3-15-1 architecture, featuring three input nodes for the tuple (x_1, x_2, q) and a hidden layer of 15 neurons with ReLU activation. To demonstrate the efficiency of our framework, we initialize the training with a set of points: 40 from D_0 , 90 from D_u , and 180 from $D_{\mathcal{X}}$. The CEGIS loop converges in 110.95 seconds, successfully producing a valid neural certificate in appendix.

For comparison, the state-triplet method [36] synthesizes a barrier certificate for the relevant simple path in 1.13 seconds under a degree-2 polynomial template, yielding $B(x_1, x_2) = -x_1 - x_2 - 3.125x_1x_2 + 2x_1^2 + 2x_2^2$.

6.2. Two-Room Temperature

The second benchmark focuses on the formal verification of an interconnected temperature control system, denoted by $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, with its dynamical model adapted from [2]. The system operates within a state space $\mathcal{X} = [20, 34]^2$, originating from the initial configuration $\mathcal{X}_0 = [21, 24]^2$. The underlying LTL specification pertains to the target region $\mathcal{X}_{VF} = [20, 26]^2$. Specifically, the evolution of the system state is governed by the following discrete-time dynamics f :

$$f(x_1, x_2) = \begin{bmatrix} 1 - 2\alpha - \theta - \mu u(x_1) & \alpha \\ \alpha & 1 - 2\alpha - \theta - \mu u(x_2) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \mu T_h \begin{bmatrix} u(x_1) \\ u(x_2) \end{bmatrix} + \theta \begin{bmatrix} T_e \\ T_e \end{bmatrix}, \quad (39)$$

where the constant parameters are assigned as $\alpha = 0.004$, $\theta = 0.01$, $\mu = 0.15$, $T_h = 40$, and $T_e = 0$. The state-dependent control term is defined as $u(x_i) = 0.59 - 0.011x_i$ for $i \in \{1, 2\}$.

Consider the set of atomic propositions $AP = \{a, b\}$, where the labeling function is defined as $\mathfrak{L}(x_1, x_2) = \{a\}$ for $(x_1, x_2) \in \mathcal{X}_0$, and $b \in \mathfrak{L}(x_1, x_2)$ for $(x_1, x_2) \in \mathcal{X}_{VF}$. The system is required to satisfy the LTL specification $\phi = a \Rightarrow \mathbf{FG}\neg b$. The corresponding NBA, which captures the negation of the specification (i.e., $\neg\phi = a \wedge \mathbf{GF}b$), is illustrated in Figure 2. Since q_1 is the only accepting-transition start state and the initial region $\mathcal{X}_0 = [21, 24]^2$ is entirely labeled $\{a\}$ by $\mathfrak{L}$, the automaton transitions to q_1 at the very first step from any initial product state. This makes q_1 immediately reachable, so a transition safety certificate for q_1 cannot be found. We therefore apply a transition persistence certificate to bound the number of times the accepting transitions from q_1 can fire.

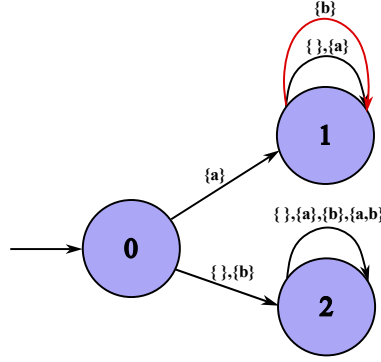


Figure 2: NBA for $a \wedge \mathbf{GF}b$ (negation of $a \Rightarrow \mathbf{FG}\neg b$). Accepting transitions (red) are $(q_1, \{b\}, q_1)$ and $(q_1, \{a, b\}, q_1)$.

Consider the transition persistence certificate template (see Appendix B). We collect 16 samples for D_0 and 196 for $D_{\mathcal{X}}$. We obtain a valid transition persistence certificate

$$B_{1,1}(x_1, x_2) = 27I_{[20,26]^2}(x_1, x_2) - x_1 I_{[20,26]^2}(x_1, x_2)$$

and $\epsilon = 0.25$ in 2.68 seconds.

We apply the neural CEGIS to synthesize $B_{1,1}(x_1, x_2)$ for $Acc^{1,1}$, using a neural network template of architecture 2-3-1 (2 inputs for (x_1, x_2) , 3 hidden neurons with squared activation, 1 output). We collect 100 samples for $D_{\mathcal{X}}$ and 16 samples for D_0 . The entire CEGIS loop terminates in 1461.91 seconds and the valid certificates obtained are provided in the appendix.

In contrast, the state-triplet method [36] times out on this benchmark. The accepting path cannot be blocked by a barrier certificate within the time limit, because the initial region is entirely contained within the target region $\mathcal{X}_{VF}$, making the accepting automaton state reachable and the state-triplet's path-blocking obligation too conservative.

6.3. Comparison

We primarily compare our approach with the state-triplet method [36] and closure certificates [25]. The state-triplet method is a foundational automata-theoretic approach that constructs barrier certificates to block all simple paths from initial states to accepting states in the automaton, requiring enumeration of state triplets along these paths. Closure certificates were specifically developed to provide more general verification conditions than the state-triplet method, and neural closure certificates [26] extend this idea with neural network templates. The Two-Room Temperature benchmark concretely illustrates the advantage over the state-triplet method. The NBA contains an accepting transition from q_1 to q_1 labeled by $\{b\}$ and $\{a, b\}$, and because the labeling function maps the entire initial region $\mathcal{X}_0 = [21, 24]^2$ to $\{a\}$, the automaton state q_1 is immediately reachable upon entering the product system. Consequently, the state-triplet approach, which requires blocking all paths to accepting states via barrier certificates, is structurally inapplicable here since q_1 cannot be made unreachable, leading to a timeout. Our transition persistence certificate $B_{1,1}$ resolves this by bounding the number of times the accepting transition can fire, without requiring q_1 to be unreachable. The core idea of closure certificates lies in characterizing an over-approximation of the transition closure, so closure certificates for LTL verification are defined over $\mathcal{X} \times \mathcal{X} \times \mathcal{Q} \times \mathcal{Q}$. In contrast, our method employs transition safety certificates to directly filter out unreachable automaton states or uses transition persistence certificates to prove that the accepting transitions

Table 1

Synthesis time comparison for polynomial-template methods (seconds)

Benchmark	Transition Certificates	State Triplet	Closure Certificate
Kuramoto 1D	1.76	0.10	Timeout
Kuramoto 2D	1.72	1.13	Timeout
Two-Room Temperature	2.68	Timeout	Timeout

Table 2

Synthesis time comparison (seconds)

Benchmark	Transition Certificates	Neural Closure Certificate
Kuramoto 1D	6.81	Timeout
Kuramoto 2D	110.95	Timeout
Two-Room Temperature	1461.91	Timeout

occur finitely often, with transition safety certificates defined over $\mathcal{X} \times \mathcal{Q}$ and transition persistence certificates defined over $\mathcal{X}$ alone. This reduces the effective certificate domain from the closure certificate's $\mathcal{X} \times \mathcal{X} \times \mathcal{Q} \times \mathcal{Q}$ to at most $\mathcal{X} \times \mathcal{Q}$, and improves synthesis efficiency. For neural templates, we adopt the CEGIS framework, whereas the neural closure certificate approach relies on sampling and posteriori verification: after sampling the state space with a given density ϵ and safety margin η , the method trains until the loss converges to zero and then performs a posteriori verification to determine whether the synthesized certificate is valid.

We conduct two groups of comparisons: (1) polynomial-template CEGIS versus the state-triplet method and closure certificates, and (2) neural-template CEGIS versus neural closure certificates. The detailed experimental configurations are provided in Appendix B. Table 1 and Table 2 report only locally reproduced, formally verified results under a five-hour timeout. For the CC and NCC baselines, the implementations follow the original synthesis-and-verification schemes; previous guarded-transition or interval-overapproximation variants are not counted in these tables. “Timeout” means that no formally verified certificate was produced within the timeout. The original closure-certificate paper reports successful CC synthesis on these benchmarks, but until the corresponding faithful local rerun produces a verified certificate, the reproduced entry is recorded as Timeout.

7. Conclusion

In this paper, we have proposed transition certificates as an efficient framework for verifying Linear Temporal Logic (LTL) properties of discrete-time dynamical systems with continuous state spaces. Our approach leverages transition safety and persistence certificates to provide greater flexibility compared to state-triplet methods [36] and to reduce search spaces relative to closure certificates [25], achieving significant speedups in LTL verification. Experimental results show that on benchmarks where the state-triplet method is structurally inapplicable due to reachable accepting states, our transition persistence certificate successfully verifies the property, while on benchmarks where state-triplet succeeds, our method achieves comparable performance with additional flexibility. We have developed automated synthesis methods based on polynomial and neural network templates, demonstrating practicality through case studies like the Kuramoto oscillator and two-room temperature model. Nonetheless, our method faces a few shortcomings that motivate future work. First, the synthesis process, particularly for neural network templates, can be computationally demanding for high-dimensional systems with highly nonlinear dynamics, due to the bottleneck in SMT solving. Second, constructing an NBA from an LTL formula has worst-case exponential complexity in the size of the formula; however, modern translators such as Spot [12] and Owl [21] produce compact automata for most practical specifications, so this is rarely a bottleneck in practice. Additionally, the current work focuses on discrete-time systems, leaving continuous-time systems as an open challenge.

References

- [1] Anand, M., Lavaei, A., Zamani, M., 2022. From small-gain theory to compositional construction of barrier certificates for large-scale stochastic systems. *IEEE Transactions on Automatic Control* 67, 5638–5645.
- [2] Anand, M., Lavaei, A., Zamani, M., 2024. Compositional synthesis of control barrier certificates for networks of stochastic systems against ω -regular specifications. *Nonlinear Analysis: Hybrid Systems* 51, 101427.

- [3] Baheti, R., Gill, H., 2011. Cyber-physical systems. The impact of control technology 12, 161–166.
- [4] Baier, C., Katoen, J.P., 2008. Principles of model checking. MIT press.
- [5] Chakarov, A., Sankaranarayanan, S., 2013. Probabilistic program analysis with martingales, in: Computer Aided Verification: 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13–19, 2013. Proceedings 25, Springer. pp. 511–526.
- [6] Chatterjee, K., Fu, H., Novotný, P., Hasheminezhad, R., 2016. Algorithmic analysis of qualitative and quantitative termination problems for affine probabilistic programs, in: Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, pp. 327–342.
- [7] Chatterjee, K., Goharshady, A., Goharshady, E., Karrabi, M., Žikelić, Đ., 2024a. Sound and complete witnesses for template-based verification of ltl properties on polynomial programs, in: International Symposium on Formal Methods, Springer. pp. 600–619.
- [8] Chatterjee, K., Goharshady, E.K., Novotný, P., Žikelić, Đ., 2024b. Equivalence and similarity refutation for probabilistic programs. Proceedings of the ACM on Programming Languages 8, 2098–2122.
- [9] Coogan, S., Gol, E.A., Arcak, M., Belta, C., 2015. Traffic network control from temporal logic specifications. IEEE Transactions on Control of Network Systems 3, 162–172.
- [10] De Moura, L., Björner, N., 2008. Z3: An efficient smt solver, in: International conference on Tools and Algorithms for the Construction and Analysis of Systems, Springer. pp. 337–340.
- [11] Ding, X., Smith, S.L., Belta, C., Rus, D., 2014. Optimal control of markov decision processes with linear temporal logic constraints. IEEE Transactions on Automatic Control 59, 1244–1257.
- [12] Duret-Lutz, A., Lewkowicz, A., Fauchille, A., Michaud, T., Renault, É., Xu, L., 2016. Spot 2.0 — a framework for ltl and omega-automata manipulation, in: Artho, C., Legay, A., Peled, D. (Eds.), Automated Technology for Verification and Analysis, Springer International Publishing, Cham. pp. 122–129.
- [13] Fainekos, G.E., Girard, A., Kress-Gazit, H., Pappas, G.J., 2009. Temporal logic motion planning for dynamic robots. Automatica 45, 343–352.
- [14] Gao, S., Kong, S., Clarke, E.M., 2013. dreal: An smt solver for nonlinear theories over the reals, in: International conference on automated deduction, Springer. pp. 208–214.
- [15] Giacobbe, M., Kroening, D., Pal, A., Tautschnig, M., 2025. Let a neural network be your invariant, in: The Thirty-ninth Annual Conference on Neural Information Processing Systems.
- [16] Hahn, E.M., Perez, M., Schewe, S., Somenzi, F., Trivedi, A., Wojtczak, D., 2020. Reward shaping for reinforcement learning with omega-regular objectives. arXiv preprint arXiv:2001.05977 .
- [17] Henzinger, T.A., Ho, P.H., Wong-Toi, H., 1997. Hytech: A model checker for hybrid systems, in: International conference on computer aided verification, Springer. pp. 460–463.
- [18] Hosseini, M., Lomuscio, A., Paoletti, N., 2025. Ltl verification of memoryful neural agents. arXiv preprint arXiv:2503.02512 .
- [19] Khaled, M., Zamani, M., 2021. Omegathreads: symbolic controller design for ω -regular objectives, in: Proceedings of the 24th International Conference on Hybrid Systems: Computation and Control, pp. 1–7.
- [20] Kress-Gazit, H., Fainekos, G.E., Pappas, G.J., 2009. Temporal-logic-based reactive mission and motion planning. IEEE transactions on robotics 25, 1370–1381.
- [21] Křetínský, J., Meggendorfer, T., Sickert, S., 2018. Owl: A library for omega-words, automata, and ltl, in: Lahiri, S.K., Wang, C. (Eds.), Automated Technology for Verification and Analysis, Springer International Publishing, Cham. pp. 543–550.
- [22] Lahijanian, M., Andersson, S.B., Belta, C., 2011. Temporal logic motion planning and control with probabilistic satisfaction guarantees. IEEE Transactions on Robotics 28, 396–409.
- [23] Leahy, K., Cristofalo, E., Vasile, C.I., Jones, A., Montijano, E., Schwager, M., Belta, C., 2019. Control in belief space with temporal logic specifications using vision-based localization. The International Journal of Robotics Research 38, 702–722.
- [24] Murali, V., Oumer, M.A., Zamani, M., 2025. Control closure certificates, in: International Symposium on Automated Technology for Verification and Analysis, Springer. pp. 387–407.
- [25] Murali, V., Trivedi, A., Zamani, M., 2024. Closure certificates, in: Proceedings of the 27th ACM International Conference on Hybrid Systems: Computation and Control, pp. 1–11.
- [26] Nadali, A., Murali, V., Trivedi, A., Zamani, M., 2024. Neural closure certificates, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 21446–21453.
- [27] Pnueli, A., 1977. The temporal logic of programs, in: 18th annual symposium on foundations of computer science (sfcs 1977), IEEE. pp. 46–57.
- [28] Podolski, A., Rybalchenko, A., 2004. Transition invariants, in: Proceedings of the 19th Annual IEEE Symposium on Logic in Computer Science, 2004., IEEE. pp. 32–41.
- [29] Prajna, S., Jadbabaie, A., 2004. Safety verification of hybrid systems using barrier certificates, in: International workshop on hybrid systems: Computation and control, Springer. pp. 477–492.
- [30] Rajkumar, R., Lee, I., Sha, L., Stankovic, J., 2010. Cyber-physical systems: the next computing revolution, in: Proceedings of the 47th design automation conference, pp. 731–736.
- [31] Rungger, M., Zamani, M., 2016. Scots: A tool for the synthesis of symbolic controllers, in: Proceedings of the 19th international conference on hybrid systems: Computation and control, pp. 99–104.
- [32] Solar-Lezama, A., 2008. Program synthesis by sketching. University of California, Berkeley.
- [33] Tabuada, P., 2009. Verification and control of hybrid systems: a symbolic approach. Springer Science & Business Media.
- [34] Vasile, C.I., Leahy, K., Cristofalo, E., Jones, A., Schwager, M., Belta, C., 2016. Control in belief space with temporal logic specifications, in: 2016 IEEE 55th Conference on Decision and Control (CDC), IEEE. pp. 7419–7424.
- [35] Wang, P., Bai, J., Zhi, D., Zhang, M., Ong, L., . Verifying omega-regular properties of neural network-controlled systems via proof certificates, in: ICLR 2025 Workshop: VeriFAI: AI Verification in the Wild.
- [36] Wongpiromsarn, T., Topcu, U., Lamperski, A., 2016. Automata theory meets barrier certificates: Temporal logic verification of nonlinear systems. IEEE Transactions on Automatic Control 61, 3344–3355.

- [37] Wongpiromsarn, T., Topcu, U., Murray, R.M., 2009. Receding horizon temporal logic planning for dynamical systems, in: Proceedings of the 48th IEEE Conference on Decision and Control (CDC) held jointly with 2009 28th Chinese Control Conference, IEEE. pp. 5997–6004.
- [38] Zhao, H., Zeng, X., Chen, T., Liu, Z., 2020. Synthesizing barrier certificates using neural networks, in: Proceedings of the 23rd international conference on hybrid systems: Computation and control, pp. 1–11.

A. Proof

Lemma A.1. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a trajectory $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow \Sigma$ and an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc)$, if a transition safety certificate $B_k(x, q)$ with respect to $q_k \in Acc^{start}$ can be found, then it can be guaranteed that all runs of all words in $\mathfrak{L}(\mathfrak{S})$ cannot visit the state q_k .*

Proof. By Definition 4.1, we have $\mathcal{Y}_u = \{(x, q_k) \mid x \in \mathcal{X}\}$ as the unsafe set in the product system $\mathfrak{S} \times \mathcal{A}$. We verify that B_k satisfies the barrier certificate conditions (4, 5, 6) with respect to $\mathcal{Y}_u$:

- Condition (10) states $B_k(x_0, q_0) \geq 0$ for all $(x_0, q_0) \in \mathcal{Y}_0$, which corresponds to condition (4).
- Condition (11) states $B_k(x, q_k) < 0$ for all $(x, q_k) \in \mathcal{Y}_u$, which corresponds to condition (5).
- Condition (12) states $B_k(x, q) \geq 0 \Rightarrow B_k(x', q') \geq 0$ for all $(x', q') \in F(x, q)$, which corresponds to condition (6).

By Theorem (3.2), any trajectory of the product system $\mathfrak{S} \times \mathcal{A}$ cannot visit $\mathcal{Y}_u$.

Now we establish the connection to automaton runs. For any word $w = \mathfrak{L}(\tau) \in \mathfrak{L}(\mathfrak{S})$ where $\tau = \langle x_0, x_1, \dots \rangle$ is a trajectory of $\mathfrak{S}$, the corresponding run r of w on $\mathcal{A}$ is a sequence $r = \langle (r_0, w_0, r_1), (r_1, w_1, r_2), \dots \rangle$ where $r_0 = q_0$. The sequence of product states $\langle (x_0, r_0), (x_1, r_1), \dots \rangle$ forms a trajectory in $\mathfrak{S} \times \mathcal{A}$. Since this trajectory cannot visit $\mathcal{Y}_u = \{(x, q_k) \mid x \in \mathcal{X}\}$, we have $r_i \neq q_k$ for all i . Therefore, runs of all words in $\mathfrak{L}(\mathfrak{S})$ cannot visit state q_k . $\square$

Lemma A.2. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc)$, and $(k, l) \in Acc^{pair}$, if a transition persistence certificate $B_{k,l}$ can be found, then it can be guaranteed that all runs of the words in $\mathfrak{L}(\mathfrak{S})$ make accepting transitions from q_k to q_l happen finitely often.*

Proof. We define $\mathcal{X}_{k,l} = \{x \mid (q_k, \mathfrak{L}(x), q_l) \in Acc^{k,l}\}$ as the set of states where accepting transitions from q_k to q_l can occur.

Consider any trajectory $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$ and its corresponding word $w = \mathfrak{L}(\tau)$. Let r be an arbitrary run of w on $\mathcal{A}$ starting from q_0 . An accepting transition from q_k to q_l at step i in run r requires two conditions: (a) the run is at state q_k at step i , i.e., $r_i = q_k$, and (b) the label enables the transition, i.e., $(q_k, \mathfrak{L}(x_i), q_l) \in Acc^{k,l}$. Since condition (b) is necessary, whenever $x_i \notin \mathcal{X}_{k,l}$ the accepting transition from q_k to q_l cannot occur at step i , regardless of which automaton state r occupies. Therefore, the number of accepting transitions from q_k to q_l in r is bounded above by the number of times the trajectory visits $\mathcal{X}_{k,l}$, and this bound holds for any run r of w on $\mathcal{A}$.

By Definition 4.4, we have:

- $B_{k,l}(x_0) \geq 0$ by condition (13).
- $B_{k,l}(x_i) \geq 0$ for all i by condition (14), since $B_{k,l}$ remains non-negative along the trajectory.
- $B_{k,l}(x_i) \geq B_{k,l}(x_{i+1}) + I_{k,l}(x_i)\epsilon$ by condition (15), where $I_{k,l}(x_i) = 1$ if $x_i \in \mathcal{X}_{k,l}$ and 0 otherwise.

Suppose the trajectory visits $\mathcal{X}_{k,l}$ at indices $i_1, i_2, \dots, i_m$ where m is the number of such visits. At each visit, $B_{k,l}$ decreases by at least ϵ :

$$B_{k,l}(x_0) \geq B_{k,l}(x_{i_1}) + \epsilon \geq B_{k,l}(x_{i_2}) + 2\epsilon \geq \dots \geq B_{k,l}(x_{i_m}) + m\epsilon. \quad (40)$$

Since $B_{k,l}(x_{i_m}) \geq 0$ by condition (14), we have $B_{k,l}(x_0) \geq m\epsilon$, which gives $m \leq \frac{B_{k,l}(x_0)}{\epsilon}$. Since the number of accepting transitions from q_k to q_l in any run is bounded above by m , they occur only finitely often. Because the bound depends only on the system trajectory and not on the choice of run, this holds for all runs of all words in $\mathfrak{L}(\mathfrak{S})$. $\square$

B. Experiment

B.1. Template for Two-Dimensional Kuramoto Oscillator System

The transition safety certificate template for the two-dimensional Kuramoto Oscillator system is given by:

$$B_0(x_1, x_2, q) = c_0 + c_1 \cdot x_1 + c_2 \cdot x_2 + c_3 \cdot q \\ + c_4 \cdot I_{X_u}(x_1, x_2) + c_5 \cdot x_1 q + c_6 \cdot x_2 q$$

B.2. Template for Two-Room Temperature System

For the *transition safety certificate*:

$$B_1(x_1, x_2, q) = c_0 + c_1 \cdot I_{X_0}(x_1, x_2) + c_2 \cdot I_{X_{VF}}(x_1, x_2) \\ + c_3 \cdot x_1 I_{X_0}(x_1, x_2) + c_4 \cdot x_2 I_{X_0}(x_1, x_2) \\ + c_5 \cdot x_1 I_{X_{VF}}(x_1, x_2) + c_6 \cdot x_2 I_{X_{VF}}(x_1, x_2) \\ + c_7 \cdot \max(x_1, x_2) + c_8 \cdot x_1^2 + c_9 \cdot x_2^2 + c_{10} \cdot q + c_{11} \cdot q^2.$$

The template for the *transition persistence certificate* is as follows:

$$B_{1,1}(x_1, x_2) = c_0 + c_1 \cdot I_{X_0}(x_1, x_2) + c_2 \cdot I_{X_{VF}}(x_1, x_2) \\ + c_3 \cdot x_1 I_{X_0}(x_1, x_2) + c_4 \cdot x_2 I_{X_0}(x_1, x_2) \\ + c_5 \cdot x_1 I_{X_{VF}}(x_1, x_2) + c_6 \cdot x_2 I_{X_{VF}}(x_1, x_2) \\ + c_7 \cdot \max(x_1, x_2) + c_8 \cdot x_1^2 + c_9 \cdot x_2^2.$$

B.3. Neural Network Certificate Synthesis Results

This section provides the neural network certificates synthesized in the three case studies.

B.3.1. Kuramoto Oscillator: One-Dimensional System

Valid transition safety certificate network parameters:

- **Network architecture:** $2 \rightarrow 10 \rightarrow 1$ (ReLU activation network)
- **Layer 1 weights** $W^{(1)} \in \mathbb{R}^{10 \times 2}$:

$$W^{(1)} = \begin{bmatrix} 0.35423541 & 0.03017079 \\ 0.40347892 & 0.15485710 \\ -0.33290595 & 0.20670444 \\ 0.05953831 & -0.57081002 \\ -0.77764583 & -0.30270880 \\ 0.28167149 & -0.56571352 \\ -0.00433345 & 0.27621469 \\ -0.11347809 & -0.37141919 \\ 0.30172166 & -0.14517429 \\ 0.00136681 & 0.32282710 \end{bmatrix}$$

- **Layer 1 bias** $b^{(1)} \in \mathbb{R}^{10}$:

$$b^{(1)} = \begin{bmatrix} -0.83490396 & 0.57956743 & -0.47568363 & -0.27932465 & -0.28890583 \\ 0.16922584 & -0.66955644 & -0.23689666 & 0.49750403 & 0.55335736 \end{bmatrix}^T$$

- **Layer 2 weights** $W^{(2)} \in \mathbb{R}^{1 \times 10}$:

$$W^{(2)} = \begin{bmatrix} -1.36676764 & 0.13470453 & -0.01585477 & -0.04601728 & 0.22054167 \\ -0.15794340 & -0.09915265 & 0.00324699 & -0.02935893 & 0.18765648 \end{bmatrix}$$

- **Layer 2 bias** $b^{(2)} \in \mathbb{R}$:

$$b^{(2)} = -0.25243226$$

- **Analytical expression:**

$$B(x, q) = W^{(2)} \cdot \text{ReLU}(W^{(1)} \cdot [x, q]^T + b^{(1)}) + b^{(2)}$$

B.3.2. Kuramoto Oscillator: Two-Dimensional System

Valid transition safety certificate network parameters:

- **Network architecture:** $3 \rightarrow 15 \rightarrow 1$ (ReLU activation network)

- **Layer 1 weights** $W^{(1)} \in \mathbb{R}^{15 \times 3}$:

$$W^{(1)} = \begin{bmatrix} -0.01797801 & 0.64069444 & -0.03856681 \\ -0.57299656 & -0.14924462 & -0.04329325 \\ -0.56604266 & -0.25653100 & 0.10918397 \\ -0.12088224 & 0.62192768 & -0.95162183 \\ -0.54926884 & -0.45272782 & 0.26691911 \\ -0.35724673 & 0.10176215 & -0.26044405 \\ -0.62156856 & 0.45307916 & -0.63630158 \\ -0.50465399 & -0.00161430 & 0.04746520 \\ -0.21175091 & 0.12297605 & -0.52375120 \\ 0.60632193 & 0.14016706 & -2.34126782 \\ -0.07868959 & -0.15929390 & -0.10671024 \\ 0.68689883 & -0.58823287 & -0.65129077 \\ -0.75197023 & 0.13900414 & -0.01829747 \\ -0.55132180 & 0.17512728 & -0.04693245 \\ -0.18292713 & 0.10734314 & -0.20697621 \end{bmatrix}$$

- **Layer 1 bias** $b^{(1)} \in \mathbb{R}^{15}$:

$$b^{(1)} = \begin{bmatrix} -0.23332441 & -0.25264311 & -0.20304658 & 0.00498547 & -0.41519192 \\ -0.18106091 & -0.32431474 & 0.83221281 & -0.54517263 & 0.07074507 \\ -0.18164203 & 0.44940028 & 0.60892761 & -0.63835907 & 0.36881346 \end{bmatrix}^T$$

- **Layer 2 weights** $W^{(2)} \in \mathbb{R}^{1 \times 15}$:

$$W^{(2)} = \begin{bmatrix} -0.22106327 & 0.12717371 & -0.04997402 & -0.50930381 & -0.18251525 \\ 0.08938225 & 0.48234484 & -0.43327656 & -0.12058055 & 0.42470711 \\ 0.11804953 & -0.65379494 & 0.25955433 & 0.05265374 & 0.01196656 \end{bmatrix}$$

- **Layer 2 bias** $b^{(2)} \in \mathbb{R}$:

$$b^{(2)} = 0.36412391$$

- **Analytical expression:**

$$B(x_1, x_2, q) = W^{(2)} \cdot \text{ReLU}(W^{(1)} \cdot [x_1, x_2, q]^T + b^{(1)}) + b^{(2)}$$

B.3.3. Two-Room Temperature System

Valid transition persistence certificate network parameters:

- **Network architecture:** $2 \rightarrow 3 \rightarrow 1$

- **Decrease constant** ϵ : 0.01 (fixed)

- **Layer 1 weights** $W^{(1)} \in \mathbb{R}^{3 \times 2}$:

$$W^{(1)} = \begin{bmatrix} -0.12892260 & 0.07935464 \\ 0.39971098 & -0.35251060 \\ -0.61775041 & 0.61788529 \end{bmatrix}$$

- **Layer 1 bias** $b^{(1)} \in \mathbb{R}^3$:

$$b^{(1)} = [1.44339669 \quad -1.41302800 \quad -0.01615771]^T$$

B.4. Comparative Method Setting

B.4.1. State-Triplet Method

The state-triplet baseline follows the accepting-state NBA path-blocking scheme of [36]. For each accepting state, all simple paths from the initial automaton state to the accepting state are enumerated. For a triplet (q_i, q_j, q_k) , let Y_0 be the union of labels on $q_i \rightarrow q_j$, Y_1 the union of labels on $q_j \rightarrow q_k$, and Y_m the labels on the self-loop $q_j \rightarrow q_j$ when present. We synthesize a barrier B satisfying

$$B(x) \leq 0 \text{ on } Y_0, \quad B(x) \geq 10^{-2} \text{ on } Y_1,$$

and the invariance condition

$$x \in (Y_0 \cup Y_m \cup Y_1) \setminus Y_1, \quad f(x) \in Y_0 \cup Y_m \cup Y_1, \quad B(x) \leq 0 \Rightarrow B(f(x)) \leq 0.$$

Candidate coefficients are synthesized by Z3 and formally checked by dReal for the Kuramoto systems and by Z3 for the two-room system.

For the 1D Kuramoto benchmark, the state-triplet template is

$$B(x) = c_0 + c_1x + c_2x^2 + c_3x^3 + c_4x^4.$$

The synthesized certificate is

$$B(x) = -4.5x^2 + 2x^3,$$

which blocks the accepting path $0 \rightarrow 1 \rightarrow 2$ with $Y_0 = \{i\}$, $Y_1 = \{u\} \cup \{i, u\}$, and $Y_m = \emptyset \cup \{i\}$. The direct accepting edge $0 \rightarrow 2$ is discharged because its label region $\{i, u\}$ is empty.

For the 2D Kuramoto benchmark, the tested state-triplet template is

$$B(x_1, x_2) = c_0 + c_1x_1 + c_2x_2 + c_3x_1x_2 + c_4x_1^2 + c_5x_2^2.$$

The direct accepting edge $0 \rightarrow 2$ is again discharged by the empty label region $\{i, u\}$. The path $0 \rightarrow 1 \rightarrow 2$ is blocked by the synthesized certificate

$$B(x_1, x_2) = -x_1 - x_2 - 3.125x_1x_2 + 2x_1^2 + 2x_2^2,$$

with $Y_0 = \{i\}$, $Y_1 = \{u\} \cup \{i, u\}$, and $Y_m = \emptyset \cup \{i\}$. This run used dReal precision 10^{-3} for the dynamics-dependent invariance check.

For the two-room benchmark, the tested state-triplet template is

$$B(x_1, x_2) = c_0 + c_1x_1 + c_2x_2 + c_3x_1x_2 + c_4x_1^2 + c_5x_2^2 + c_6 \max(x_1, x_2).$$

The accepting path $0 \rightarrow 2 \rightarrow 1$, with $Y_0 = \{a\} \cup \{a, b\}$, $Y_1 = \{b\} \cup \{a, b\}$, and $Y_m = \emptyset \cup \{a\}$, is not discharged before timeout. This is expected for this persistence-style benchmark: since $\mathcal{X}_0 \subseteq \mathcal{X}_{VF}$, traces from the initial region can satisfy both a and b , so the accepting state of the state-based NBA is reachable and the path-blocking ST obligation is too conservative.

B.4.2. Closure Certificates Method

The synthesis process uses Z3 to find candidate certificates and dReal to find counterexamples, with a dReal precision of 0.0001.

One-Dimensional Kuramoto Oscillator System: The NBA is defined as $(\Sigma, Q, q_0, \delta, \text{Acc})$ to represent Fa .

$$\text{Acc} = \{0\}$$

$$Q = \{1, 0\}$$

$$q_0 = 1$$

$$\Sigma = \{\emptyset, \{a\}\}$$

$$\delta = \{(1, \emptyset, 1), (1, \{a\}, 0), (0, \emptyset, 0), (0, \{a\}, 0)\}$$

Table 3

Current faithful CC reproduction status (One-Dimensional System)

Benchmark	Status
1D Kuramoto CC	Timeout

a is defined as $x \in X_u$

The certificate templates are:

$$T_{0,0} = c_0 + c_1x + c_2y$$

$$T_{0,1} = c_3 + c_4x + c_5y$$

$$T_{1,0} = c_6 + c_7x + c_8y$$

$$T_{1,1} = c_9 + c_{10}x + c_{11}y$$

The current faithful rerun is recorded as Timeout, so no reproduced coefficients are reported in Table 3.

Two-Dimensional Kuramoto Oscillator System: Under this template, the current faithful rerun is recorded as Timeout under the five-hour cap. The certificate template is defined for $i, j \in \{0, 1\}$ as:

$$\begin{aligned} T_{i,j}((x_1, x_2), (y_1, y_2)) = & c_{0,i,j} + c_{1,i,j}y_1I_{X_0}(x_1, x_2) + c_{2,i,j}y_2I_{X_0}(x_1, x_2) \\ & + c_{3,i,j}y_1I_{X_u}(x_1, x_2) + c_{4,i,j}y_2I_{X_u}(x_1, x_2) \\ & + c_{5,i,j}y_1 + c_{6,i,j}y_2 \end{aligned}$$

Two-Room Temperature System: Under this template, the current faithful rerun is recorded as Timeout under the five-hour cap. The NBA for the negated two-room objective $a \wedge \mathbf{GF}b$ is $(\Sigma, Q, q_0, \delta, \text{Acc})$.

$$Q = \{0, 1, 2, 3\}$$

$$\Sigma = \{\emptyset, \{a_0\}, \{a_1\}, \{a_0, a_1\}\}$$

$$q_0 = 0$$

$$\text{Acc} = \{2\}$$

$$\begin{aligned} \delta = \{ & (q_0, \{a_0\}, q_1), (q_0, \{a_1\}, q_1), (q_0, \{a_0, a_1\}, q_2), (q_0, \emptyset, q_3), \\ & (q_0, \{a_1\}, q_3), (q_1, \emptyset, q_1), (q_1, \{a_0\}, q_1), (q_1, \{a_1\}, q_2), \\ & (q_1, \{a_0, a_1\}, q_2), (q_2, \emptyset, q_2), (q_2, \{a_0\}, q_2), (q_2, \{a_1\}, q_2), \\ & (q_2, \{a_0, a_1\}, q_2), (q_3, \emptyset, q_3), (q_3, \{a_0\}, q_3), (q_3, \{a_1\}, q_3), \\ & (q_3, \{a_0, a_1\}, q_3) \} \end{aligned}$$

$$a_0 \text{ is defined as } x \in X_0$$

$$a_1 \text{ is defined as } x \in X_{VF}$$

The certificate template for this system is defined for $i, j \in \{0, 1, 2, 3\}$ as:

$$\begin{aligned} T_{i,j}((x_1, x_2), (y_1, y_2)) = & c_{0,i,j} + c_{1,i,j}x_1 + c_{2,i,j}x_2 + c_{3,i,j}y_1 + c_{4,i,j}y_2 \\ & + c_{5,i,j}\max(x_1, x_2) + c_{6,i,j}\max(y_1, y_2) \\ & + c_{7,i,j}x_1^2 + c_{8,i,j}x_2^2 + c_{9,i,j}y_1^2 + c_{10,i,j}y_2^2 \end{aligned}$$

B.4.3. Neural Closure Certificates Method

The configuration for the neural closure certificates is summarized in Table 4. The faithful NCC reruns use the original one-hidden-layer ReLU architecture and posteriori Lipschitz verification. Current certified results are recorded as Timeout in Table 2; the architecture and verification settings are summarized in Table 4.

Table 4

Neural network architecture and parameters for Neural Closure Certificates

System	Network Architecture	Activation	Lipschitz Estimation
1D Kuramoto Oscillator	(4-80-1)	ReLU	Spectral Norm
2D Kuramoto Oscillator	(6-80-1)	ReLU	Spectral Norm
Two-Room Temperature	(6-80-1)	ReLU	Spectral Norm